Authors:
- Guangheng (Jack) Tong (1129548)
- Jinhao He (1327850)

COMP30024 Artificial Intelligence Project Part A

Part 1:

- Algorithm: we used A* algorithm introduced in the week 3 because of its efficiency and optimality in shortest pathfinding problems, compared to DFS & BFS.
- Implementation details:
  - General approaches:
    - The hardest part of this project part A is to calculate the valid actions from the board:
      - My approach is to find the all the red coords first, saving them into a set, since sequence is not important at this stage.
      - Then find one valid adjacent coords to these existing red coords, saving them into a set of tuples of coords.
      - Repeat the above process three times for each tuple of coords in the above set, getting a set of tuples of 4 coords.
      - Sort them and remove the repetition (set data structure can do it automatically). Then we found the valid actions from the board.
    - My goal test method: After eliminating the filled rows and columns in the board, if the target is not in the board we have reached our goal; if the target is still in the board we have not reached our goal.
    - My heuristic function: See part 2
  - Choice of relevant data structures:
    - Use set to store all red coords.
    - Use set of tuples of coords to store actions.
    - Use Priority Queue to store frontier with the first element as the heuristic value.
    - Use a hash table to store visited cells in the board to avoid loops.
  - Time complexity
    - Find the valid actions:
      - O(Number of Red coords * 4 * 3 * 5 * 7): The first expansion of a single red existing coords has 4 possible valid adjacent coords at most; The second expansion of a the new coord reached in the above step is of at most 3 possible valid adjacent coords; The third expansion is of at most 5 possible valid adjacent coords; The fourth expansion is of at most 7 possible valid adjacent coords.
    - Goal test: O(1). We only need to test if the target is on the board. If not, the goal has been reached, output the actions.
    - Heuristic function: $O(2*N_{red} + 20) = O(N_{red})$.
      - $2*N_{red}$: check the horizontal and vertical distance from all red tokens to the column and row of the target. => to find the nearest red token.
      - 20: check all the cells in the column and row of the target. => to find the number of missing cells.
    - A* search: O(b ^ (the relative error in h * the length of solution d)):
      - b: branching factor, i.e., the number of valid actions. Here, b = at most 4*3*5*7 = 420
      - the relative error in h = (h – h*)/h*
        - h* is the true cost; h is the heuristic cost.
      - d: the depth of the shallowest goal.

Authors:
- Guangheng (Jack) Tong (1129548)
- Jinhao He (1327850)

- Space complexity
  - Find the valid actions: O(Number of Red coords * 4 * 3 * 5 * 7) tuples of 4-coords for each generation of new red coords.
  - Goal test: O(1) we do not store the results for each of the goal tests. If the test is satisfied, output the actions and end the program.
  - Heuristic function: O(1). Estimation = path so far + results of heuristic function. We store estimation as the priority value in the priority queue for A* search. Heuristic function itself does not take any extra memory space.
  - A* search: $O(b^d)$: we need to store all the nodes.
    - b: branching factor, i.e., the number of valid actions. Here, b = at most 4*3*5*7 = 420
    - d: the depth of the shallowest goal.

Part 2: Heuristic Function

Our heuristic function estimates the minimum number of tetromino placements required to remove the target Blue token. It chooses the smaller one of the following two:

- Approach A = ceiling [(the vertical distance from the nearest Red token to the target row + max(the number of the missing cells in the target row - 1, 0))/4]
- Approach B = ceiling [(the horizontal distance from the nearest Red token to the target column + max(the number of missing cells in the target column – 1, 0))/4]

Note: "I deducted 1 and compared to 0, then chose the larger one" because the potential overcounting of the cell overlapped between the estimations of horizontal/vertical distance and missing cells. "I divided by 4 and got the ceiling" because the maximum cells we can fill each time by putting a placement is 4.

The heuristic function estimates the minimum number of tetromino placements in the ideal condition, in which each cell we filled with tetromino gets us closer to the goal state. The above formula is admissible because there are only two ways to get to the goal state:

- Approach A: Reach the target row (vertical distance), then fill the target row.
- Approach B: Reach the target column (horizontal distance), then fill the target column.

Part 3: If we have to remove all Blue tokens, at most we have to repeat the A* search algorithm for every row or column that has a Blue token until we remove them all. Because it is possible to remove two or more blue tokens in the same row or column at the same time. Therefore, one of the approaches is to find the number of rows (N_{row}) and columns (N_{column}) that have a blue token; Compare these two numbers (N_{row} vs N_{column}), and find the smaller one; Then focus on eliminating the corresponding row or column.

- The time complexity would be O(N_{row} * b ^ (the relative error in h * the length of solution)) or O(N_{column} * b ^ (the relative error in h * the length of solution)), depending on the choice of eliminating row or column.
- The space complexity would be O(N_{column} * b^d) or O(N_{row} * b^d), depending on the choice of eliminating row or column.
- However, this approach is flawed and may not be optimal, since a combination of elimination of rows and columns may reach the optimal results.